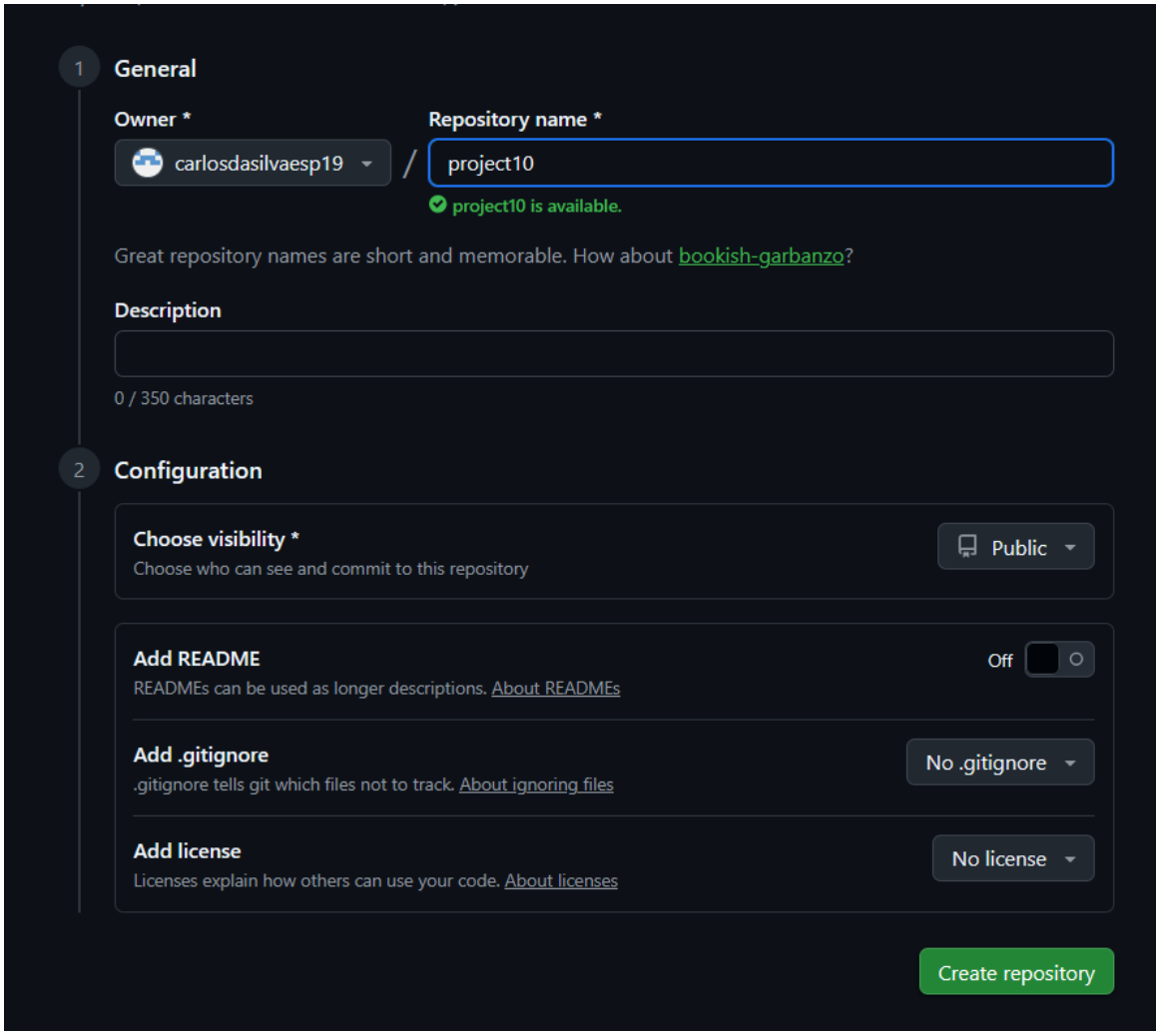
Trabajo Entornos de Desarrollo

Carlos Da Silva

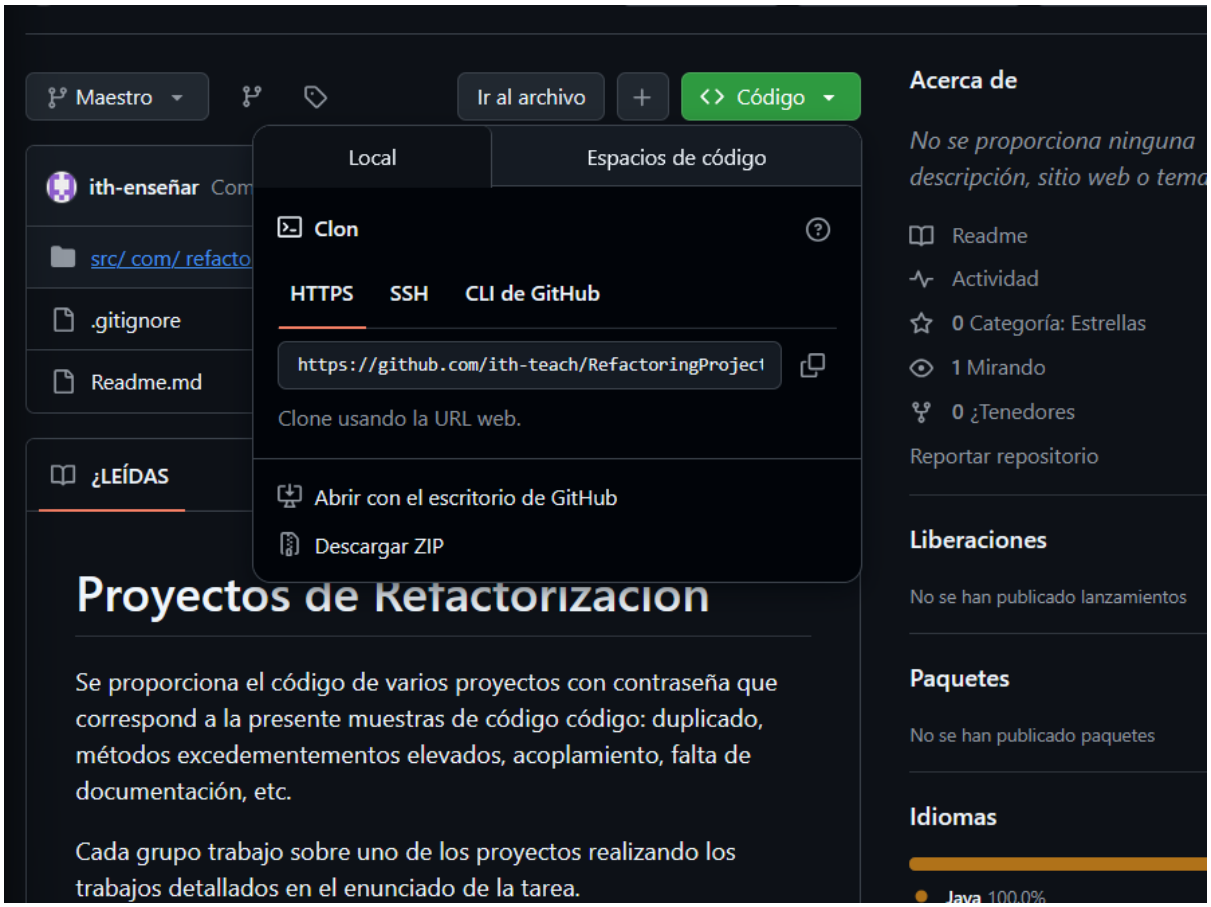
Itziar Sánchez

TAREA1

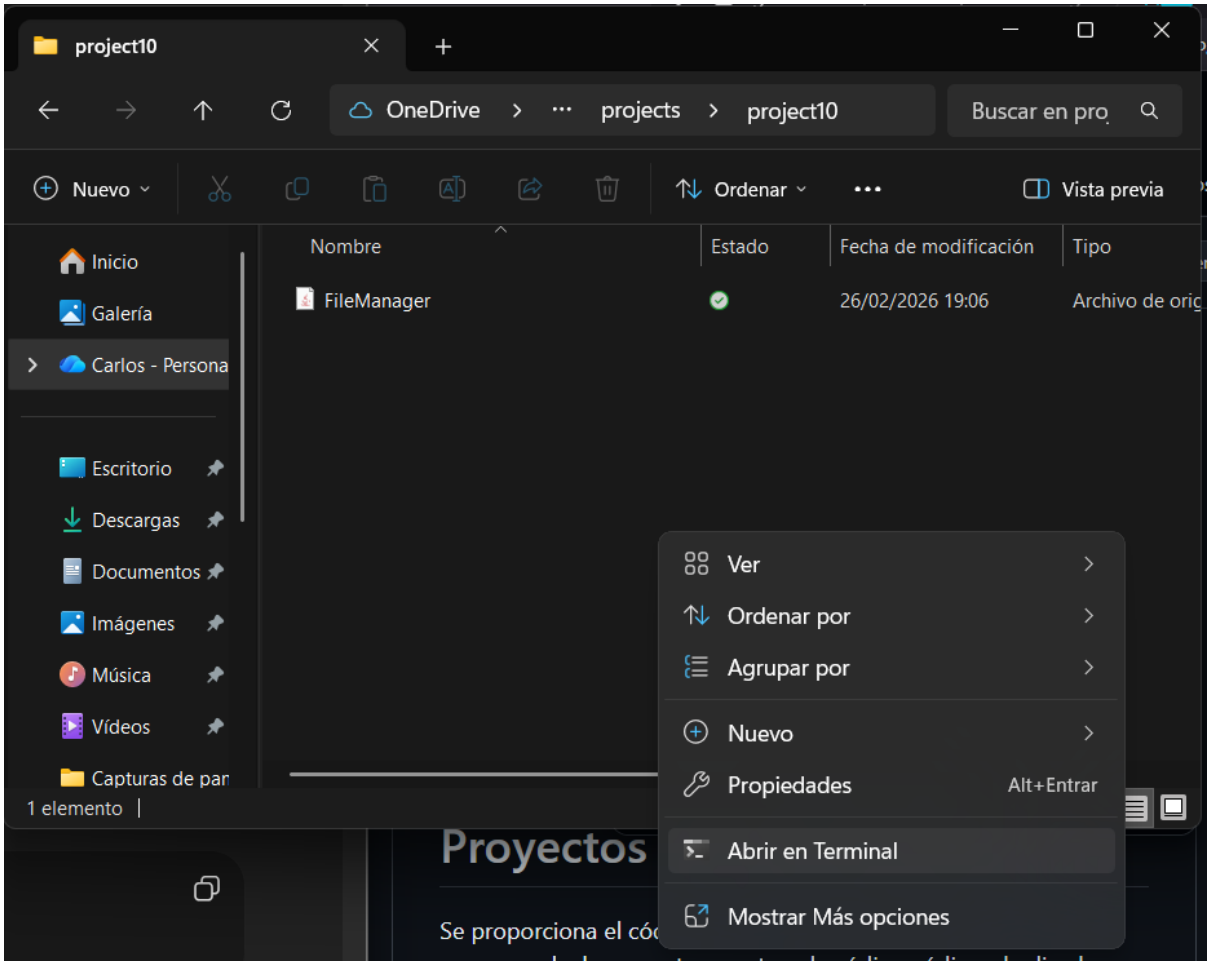
- Crear el repositorio con el nombre que indica



- Luego, descargar el repositorio en formato zip donde se encuentran los proyectos



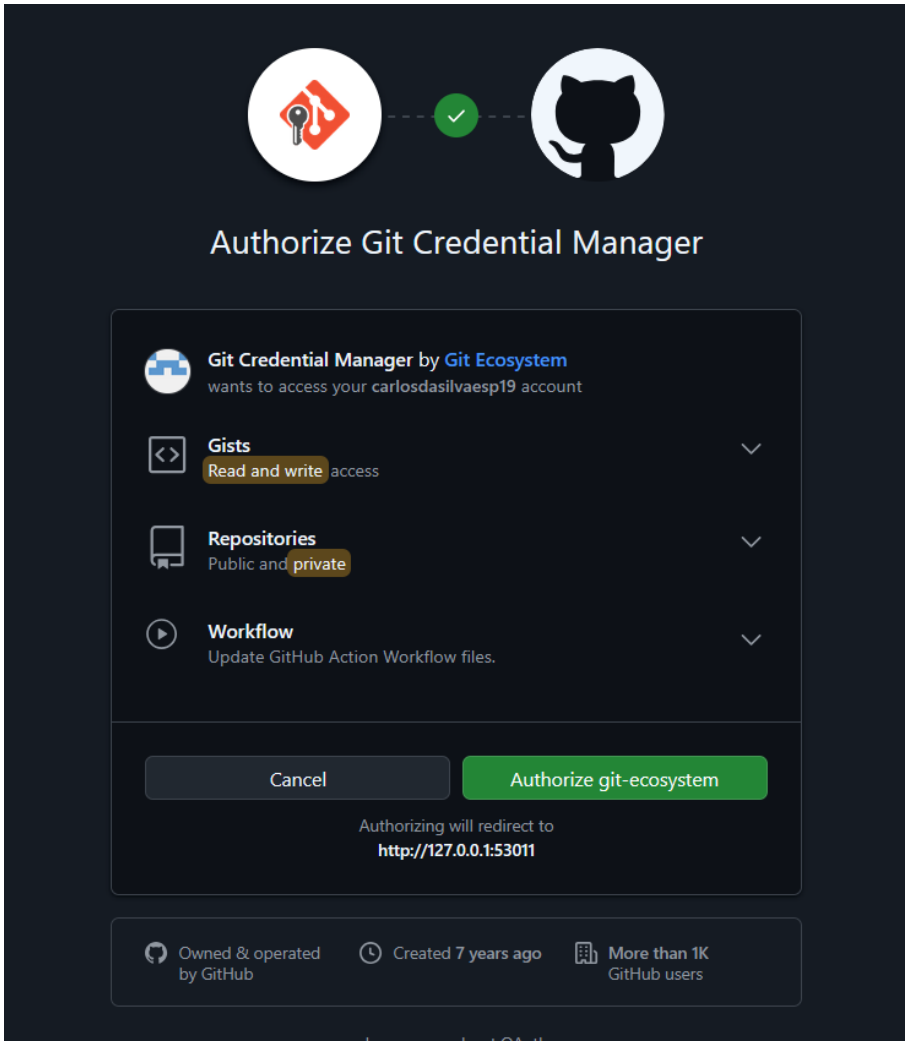
- Descomprimirlo en el Explorador de Archivos y abrir el terminal en la carpeta del proyecto indicado



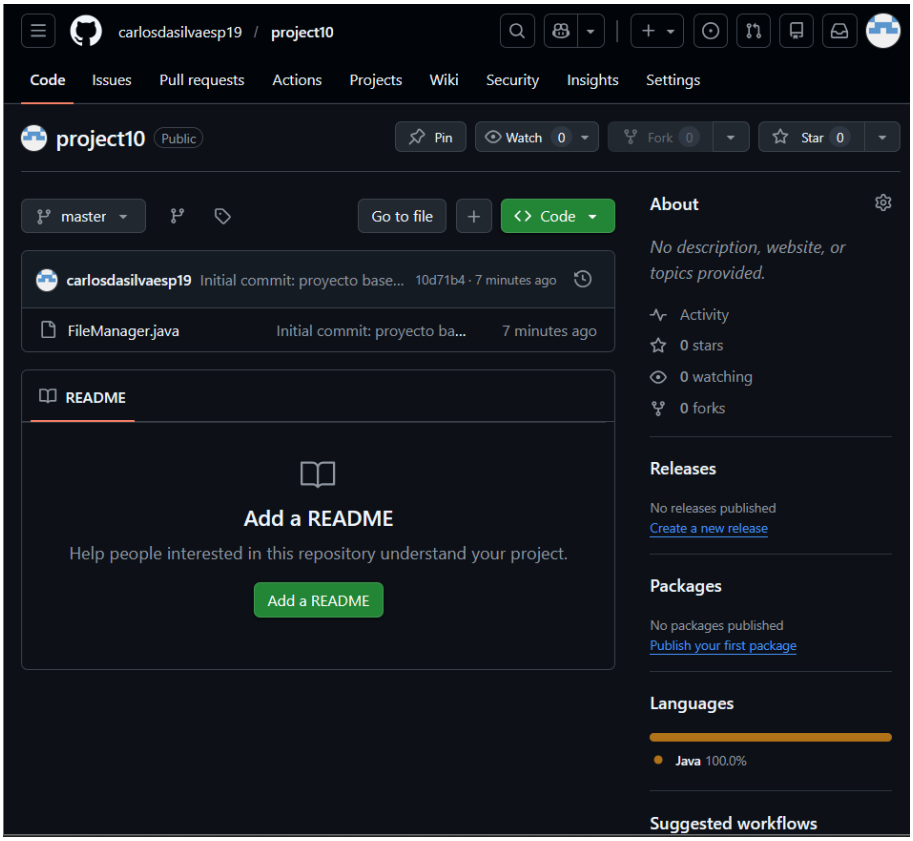
- Escribir los siguientes comandos:

```
git init

git add .

git commit -m "Initial commit: proyecto base project10"

git branch -M master

git remote add origin https://github.com/carlosdasilvaesp19/project10.git

git push -u origin master
```

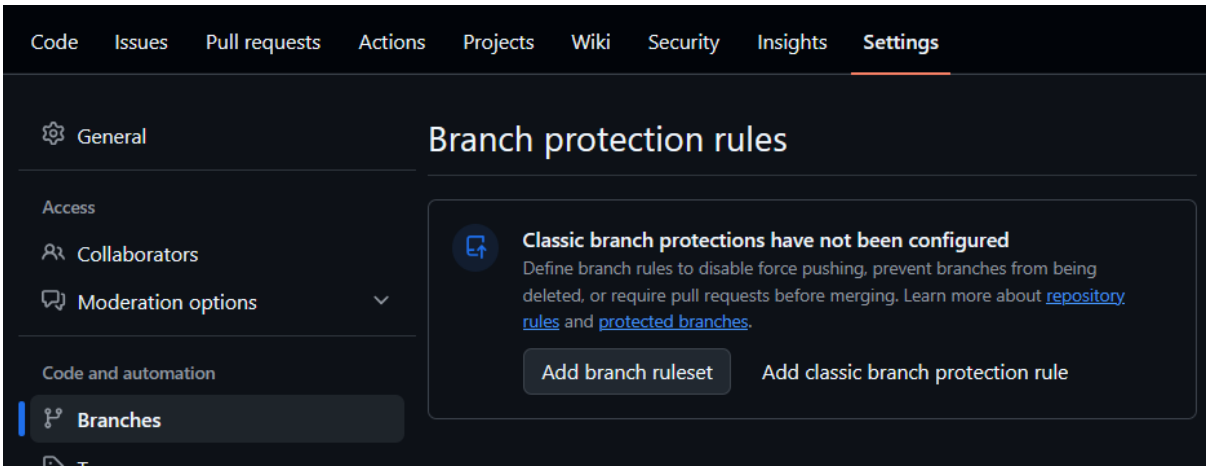
Aparecera una pestaña donde se tendra que vinvular la cuenta de git



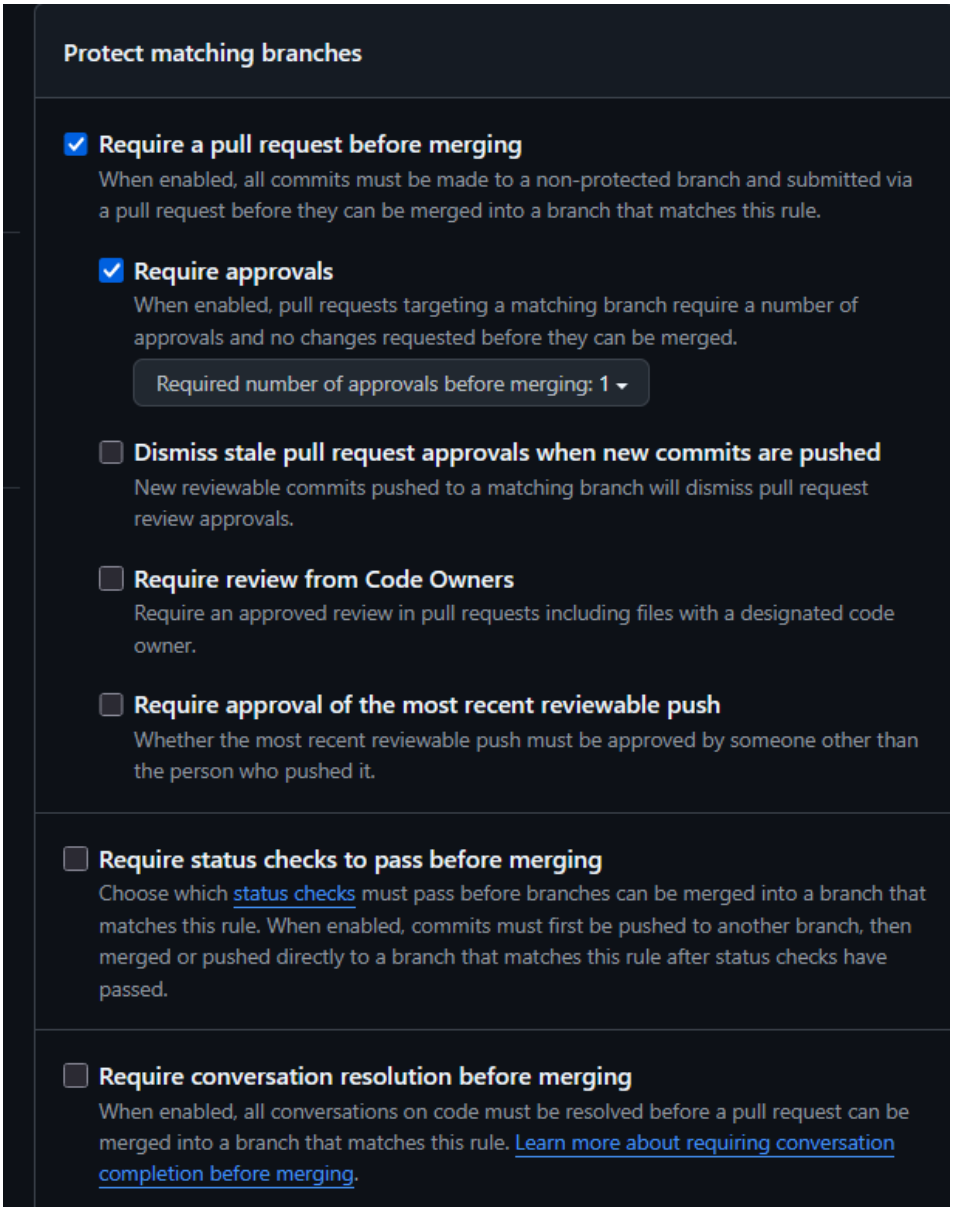
- Ir a nuestra cuenta de git y ya aparecerá master como la rama principal



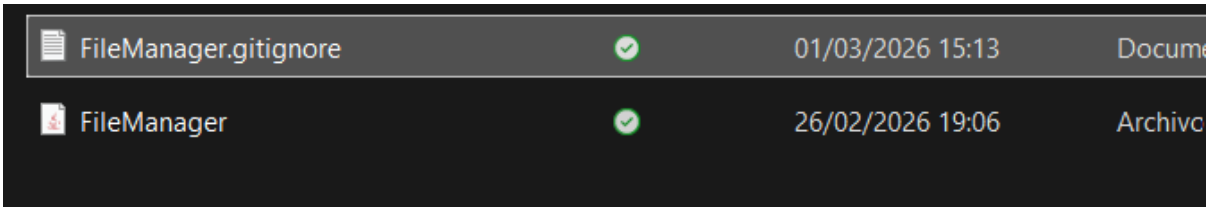
- Para poteger la rama master se va a Setting, Branches y Add classic branch protection rule



- Añadir el nombre y seleccionar las siguientes casillas y crear.Asi ya estaria protegida la rama master



- En la misma carpeta del proyecto correspondiente, crear un documento de texto llamado “.gitignore”



- Luego, abrir el terminal y escribir los siguientes comandos

```
git add .gitignore
git commit -m "chore: add .gitignore"
git push
git push --set-upstream origin feature/extract-method
```

```
PS C:\Users\carlo\OneDrive\Documentos\RefactoringProjects-master\RefactoringProjects-master\src\com\refactoring\projects
\project10> git add .gitignore
PS C:\Users\carlo\OneDrive\Documentos\RefactoringProjects-master\RefactoringProjects-master\src\com\refactoring\projects
\project10> git commit -m "chore: add .gitignore"
[feature/extract-method cc9fd57] chore: add .gitignore
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 .gitignore
PS C:\Users\carlo\OneDrive\Documentos\RefactoringProjects-master\RefactoringProjects-master\src\com\refactoring\projects
\project10> git push
fatal: The current branch feature/extract-method has no upstream branch.
To push the current branch and set the remote as upstream, use

    git push --set-upstream origin feature/extract-method

To have this happen automatically for branches without a tracking
upstream, see 'push.autoSetupRemote' in 'git help config'.

PS C:\Users\carlo\OneDrive\Documentos\RefactoringProjects-master\RefactoringProjects-master\src\com\refactoring\projects
\project10> git push --set-upstream origin /feature/extract-method
fatal: invalid refspec '/feature/extract-method'
PS C:\Users\carlo\OneDrive\Documentos\RefactoringProjects-master\RefactoringProjects-master\src\com\refactoring\projects
\project10> git push --set-upstream origin feature/extract-method
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 12 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 311 bytes | 155.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/carlosdasilvaesp19/project10.git
   10d71b4..cc9fd57  feature/extract-method -> feature/extract-method
branch 'feature/extract-method' set up to track 'origin/feature/extract-method'.
PS C:\Users\carlo\OneDrive\Documentos\RefactoringProjects-master\RefactoringProjects-master\src\com\refactoring\projects
\project10>
```

- Para crear ramas para cada patron, en la misma terminal escribir los siguientes comandos.

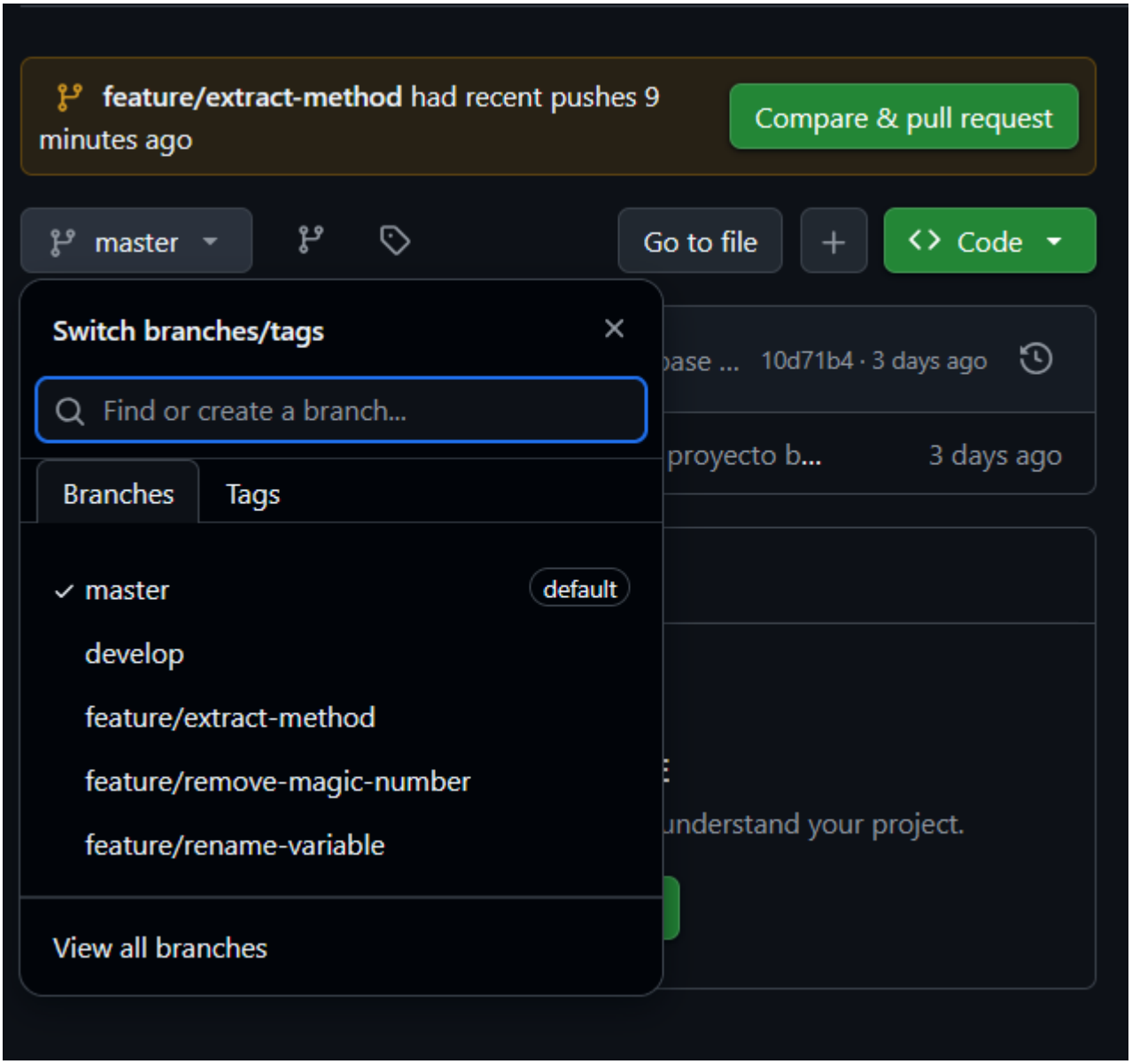
git checkout develop

git push -u origin feature/rename-variable

git checkout develop

git checkout -b feature/remove-magic-number

git push -u origin feature/remove-magic-number



- Realizar commits atomicos con mensajes:

```java
public class FileManager {
//metodo encargado de procesar un archivo
    public void procesarArchivo(String ruta) {
        File archivo = new File(ruta);
        if (archivo.exists()) {
            System.out.println("Archivo encontrado: " + archivo.getName());
            long tamano = archivo.length();
            System.out.println("Tamaño: " + tamano + " bytes");

            if (tamano > 1024 * 1024) {
                System.out.println("Archivo grande, comprimiendo...");
            }

            String extension = "";
            int i = ruta.lastIndexOf('.');
            if (i > 0) {
                extension = ruta.substring(i + 1);
            }
            System.out.println("Extensión: " + extension);
        }
    }
}
```

Y en la terminal escribimos:

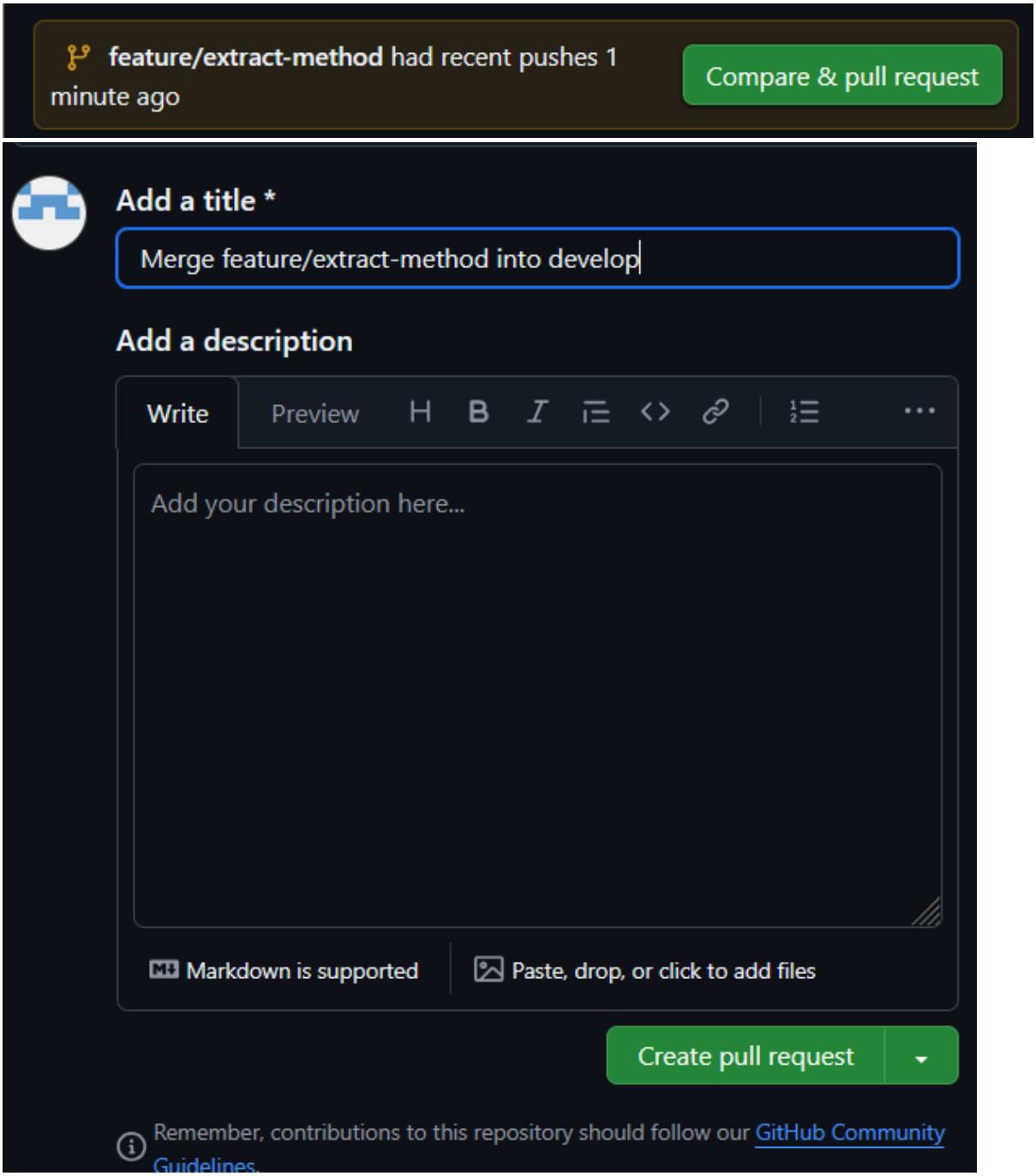
git add FileManager.java

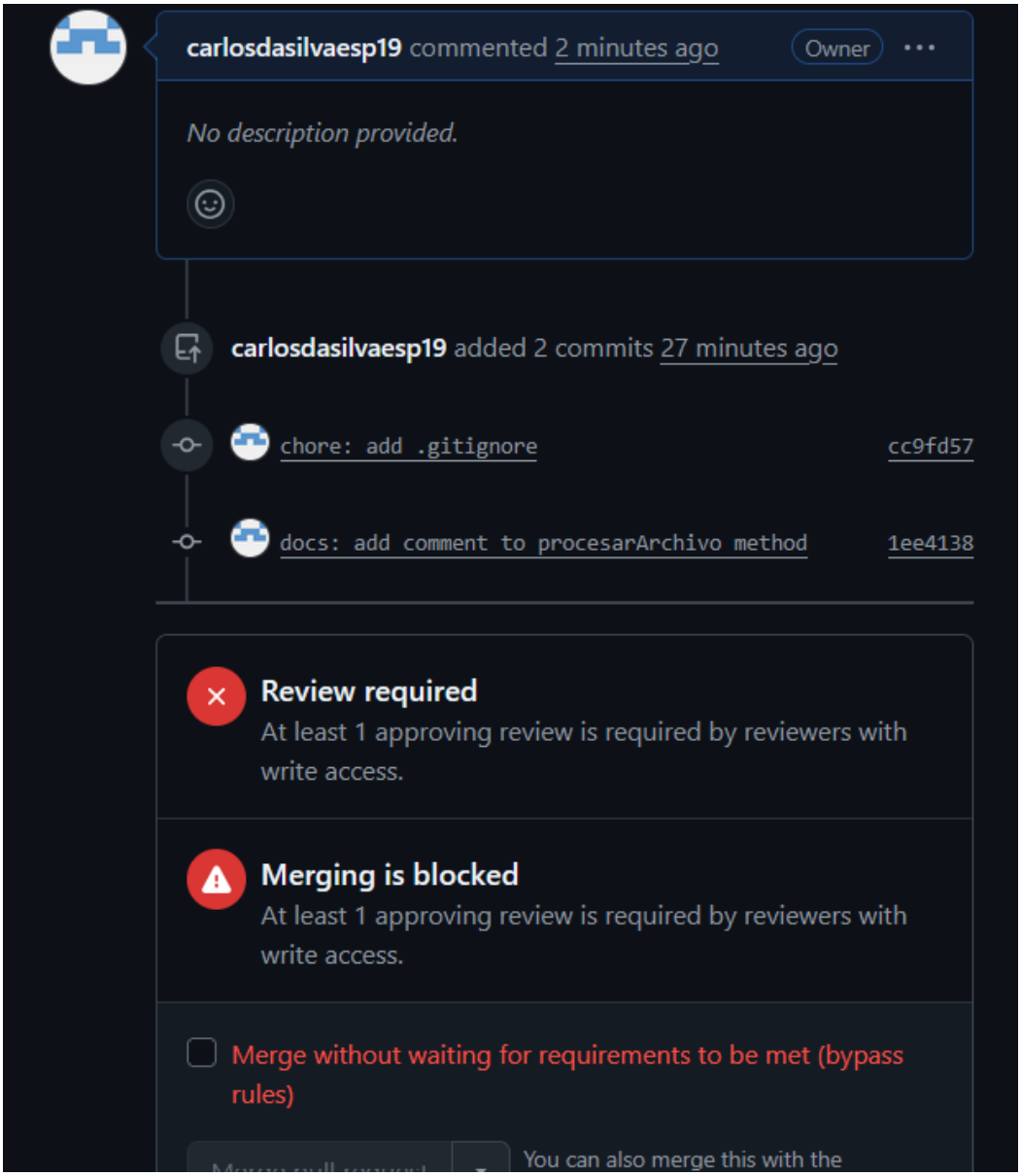
git commit -m "docs: add comment to procesarArchivo method"

git push

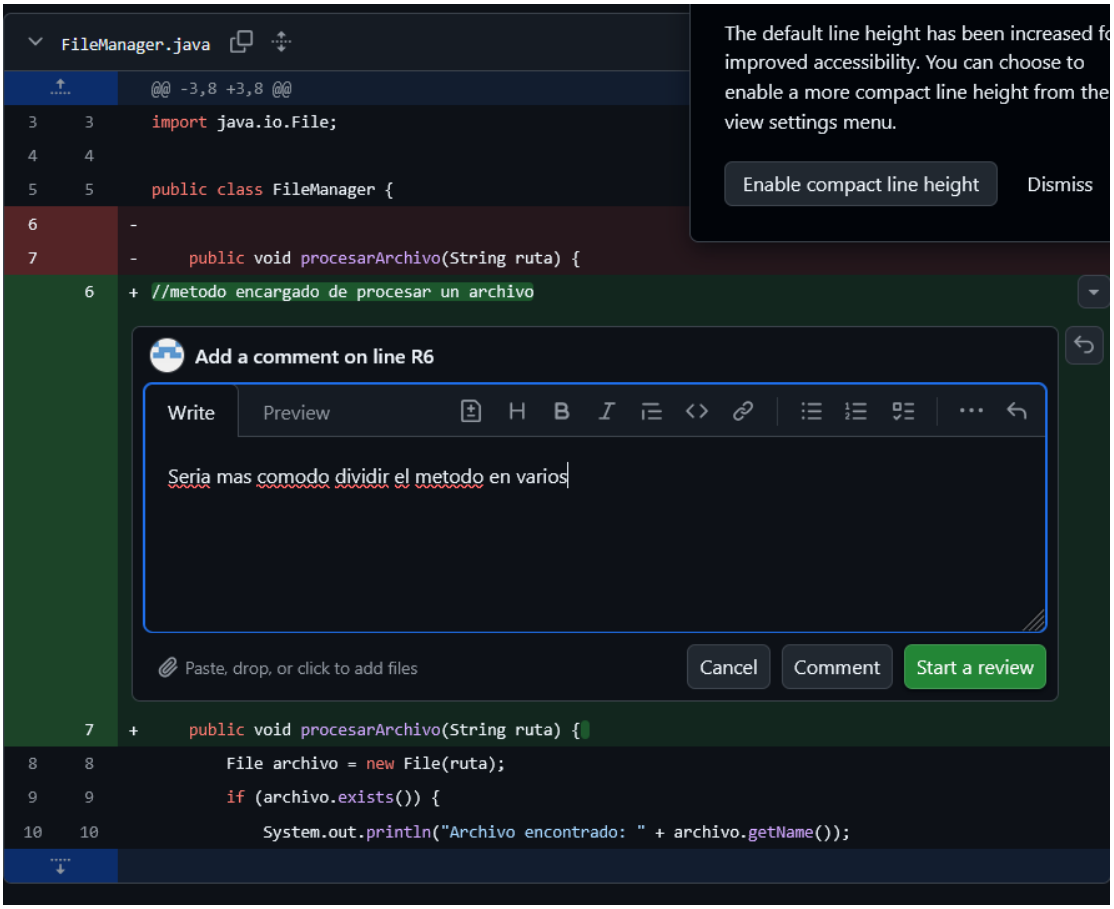
```
PS C:\Users\carlo\OneDrive\Documentos\RefactoringProjects-master\RefactoringPro
jects-master\src\com\refactoring\projects\project10> git add FileManager.java
warning: in the working copy of 'FileManager.java', LF will be replaced by CRLF
 the next time Git touches it
PS C:\Users\carlo\OneDrive\Documentos\RefactoringProjects-master\RefactoringPro
jects-master\src\com\refactoring\projects\project10> git commit -m "docs: add c
omment to procesarArchivo method"
[feature/extract-method 1ee4138] docs: add comment to procesarArchivo method
 1 file changed, 2 insertions(+), 2 deletions(-)
PS C:\Users\carlo\OneDrive\Documentos\RefactoringProjects-master\RefactoringPro
jects-master\src\com\refactoring\projects\project10> git push
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 12 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 399 bytes | 399.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/carlosdasilvaesp19/project10.git
   cc9fd57..1ee4138  feature/extract-method -> feature/extract-method
PS C:\Users\carlo\OneDrive\Documentos\RefactoringProjects-master\RefactoringPro
jects-master\src\com\refactoring\projects\project10>
```

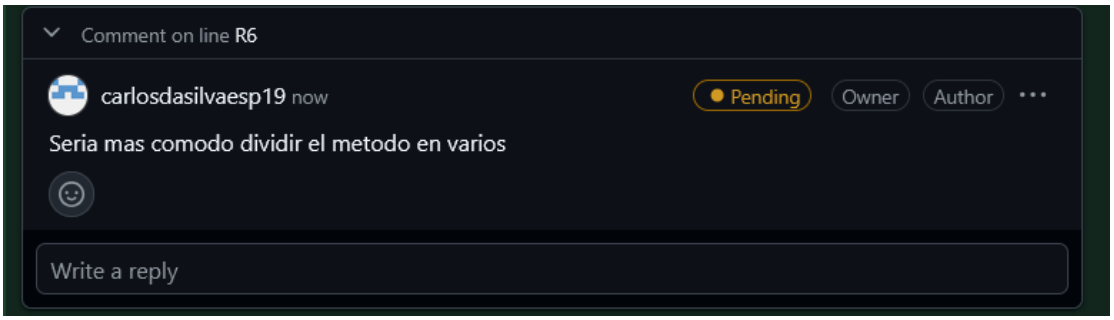
-Hacer el primer Pull Request


feature/extract-method had recent pushes 1 minute ago
Compare & pull request
Add a title *
Merge feature/extract-method into develop
Add a description
Write
Preview
Add your description here...
Markdown is supported
Paste, drop, or click to add files
Create pull request
Remember, contributions to this repository should follow our GitHub Community Guidelines.

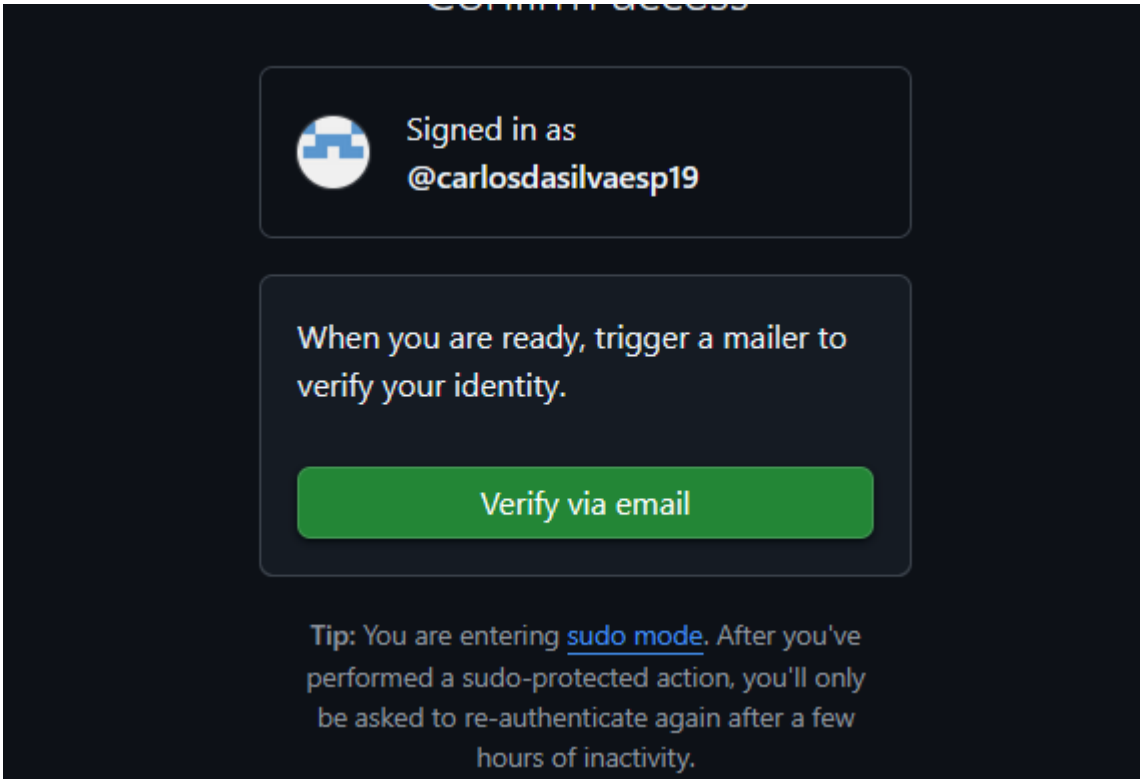


- Para simular la revisionde codigo, en el mismo Pull Request, ir a files changes, posicionarse en el + y añadir un comentario





- Añadir los miembros del grupo como colaboradores en el repositorio, se ira a settings y collaborators


Signed in as
@carlosdasilvaesp19
When you are ready, trigger a mailer to verify your identity.
Verify via email
Tip: You are entering sudo mode. After you've performed a sudo-protected action, you'll only be asked to re-authenticate again after a few hours of inactivity.

TAREA2

```java
public class FileManager {
//metodo encargado de procesar un archivo
//el metodo hace muchoas procesos
    public void procesarArchivo(String ruta) {
        File archivo = new File(ruta);
        if (archivo.exists()) {
            System.out.println("Archivo encontrado: " + archivo.getName());
            long tamano = archivo.length();
            System.out.println("Tamaño: " + tamano + " bytes");
            //el numero puede seer una constante
            if (tamano > 1024 * 1024) {
                System.out.println("Archivo grande, comprimiendo...");
            }

            String extension = "";
            //nombre de la variable no tiene sentido
            int i = ruta.lastIndexOf('.');
            if (i > 0) {
                extension = ruta.substring(i + 1);
            }
            System.out.println("Extensión: " + extension);
        }
    }
}
```

Luego subir los comentarios. En la terminal escribir:

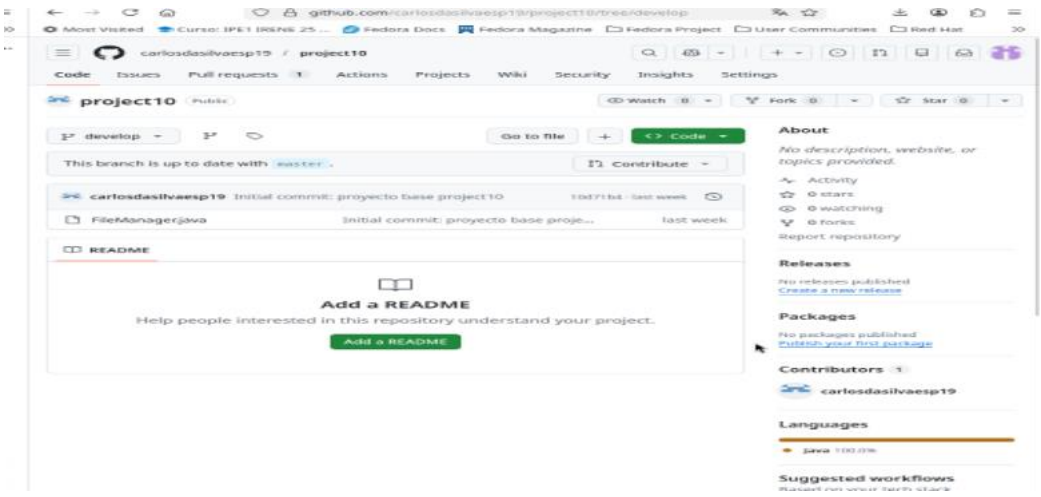
git add FileManager.java

git commit -m "docs: identify refactoring candidates"

git push

TAREA3

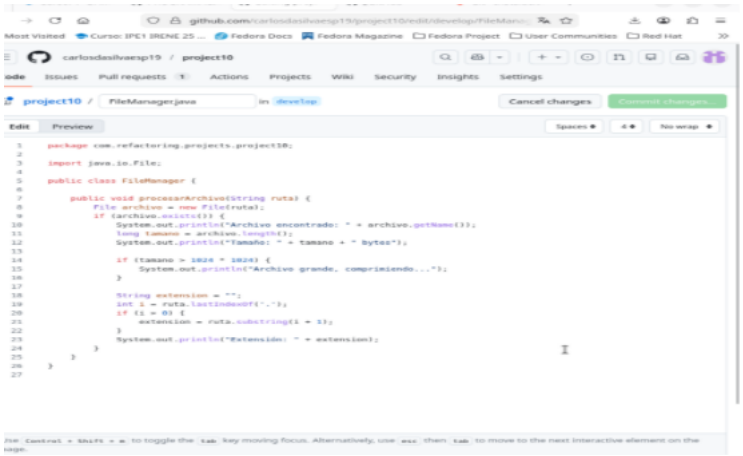
Acceder al repositorio del proyecto



Primero se accede al repositorio project10 en GitHub para visualizar el proyecto y comprobar que el archivo FileManager.java está dentro del repositorio.

Abrir el archivo FileManager.java para modificarlo

Después se abre el archivo FileManager.java desde GitHub para realizar cambios en el código del programa. En este archivo se encuentra el método procesarArchivo, que contiene la lógica para procesar un archivo y obtener su extensión.

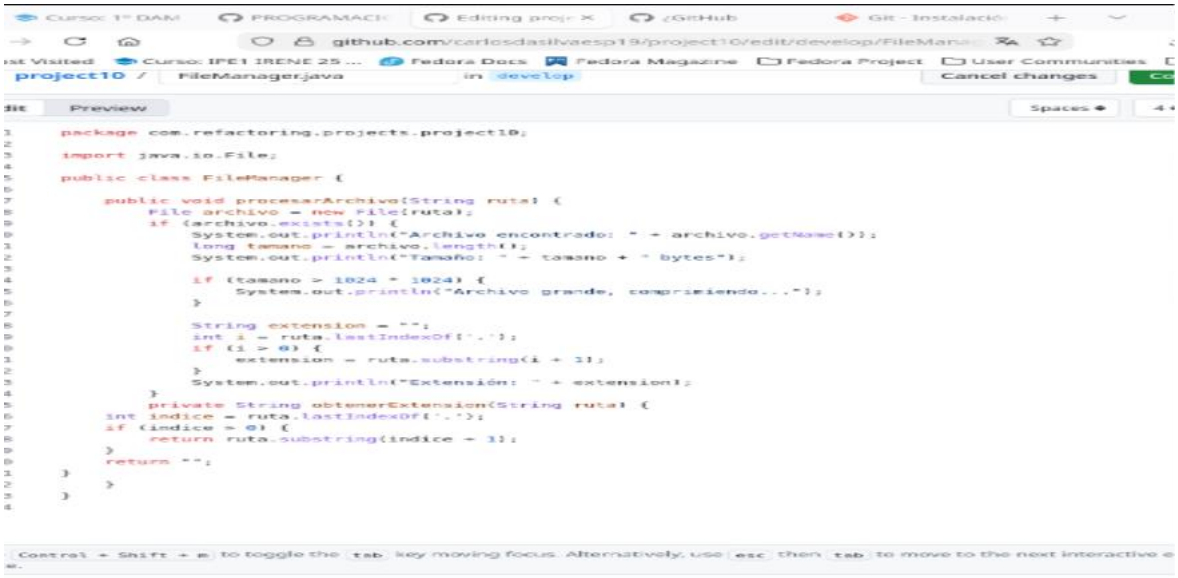


Aplicar la refactorización Extract Method

Se aplica la técnica de refactorización Extract Method, que consiste en mover una parte del código a un nuevo método para mejorar la legibilidad y reutilización.

En este caso se extrae la lógica que obtiene la extensión del archivo y se crea un nuevo método llamado: obtenerExtension(String ruta)

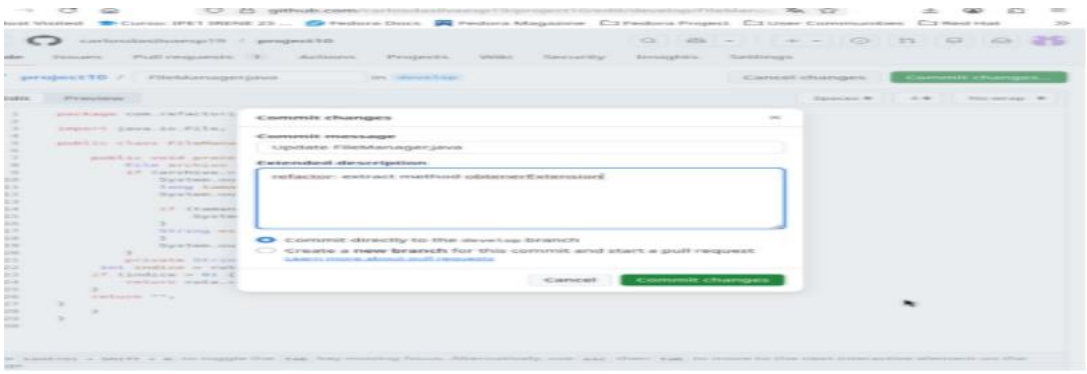
De esta manera el método procesarArchivo queda más limpio y organizado.



Guardar los cambios con un commit

Después de realizar la refactorización, se guardan los cambios realizando un commit en GitHub.

En el commit se escribe el mensaje: refactor: extract method obtenerExtension



Esto permite mantener un historial claro de los cambios realizados en el proyecto.

Crear el Pull Request

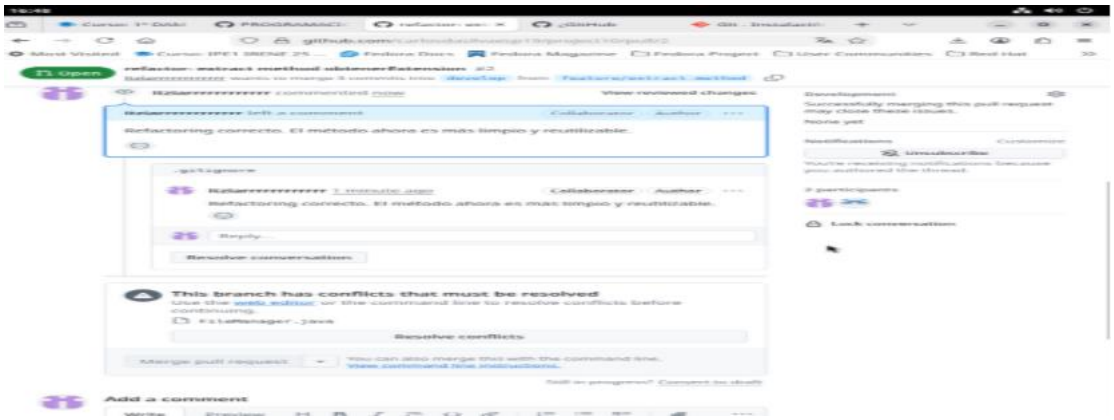
Una vez guardados los cambios, se crea un Pull Request para poder revisar e integrar la refactorización realizada en la rama de desarrollo.

El Pull Request muestra las diferencias entre las ramas y permite revisar los cambios antes de integrarlos.

Realizar la revisión del código

Finalmente se realiza una revisión del código dentro del Pull Request.

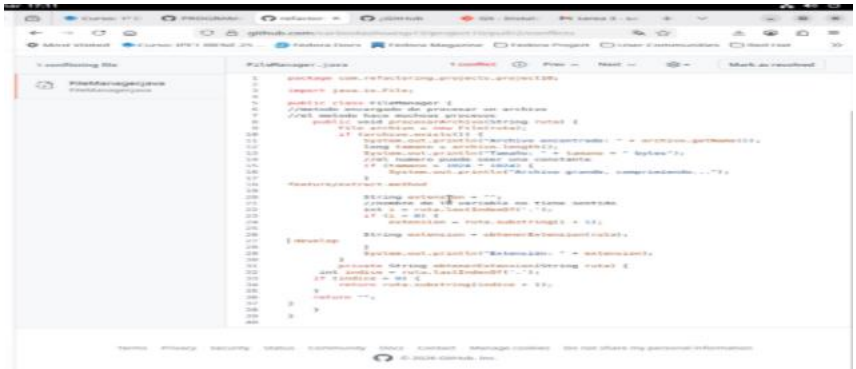
Se añade un comentario indicando que la refactorización es correcta y que el método ahora es más limpio y reutilizable.



TAREA 4

Aparición del conflicto

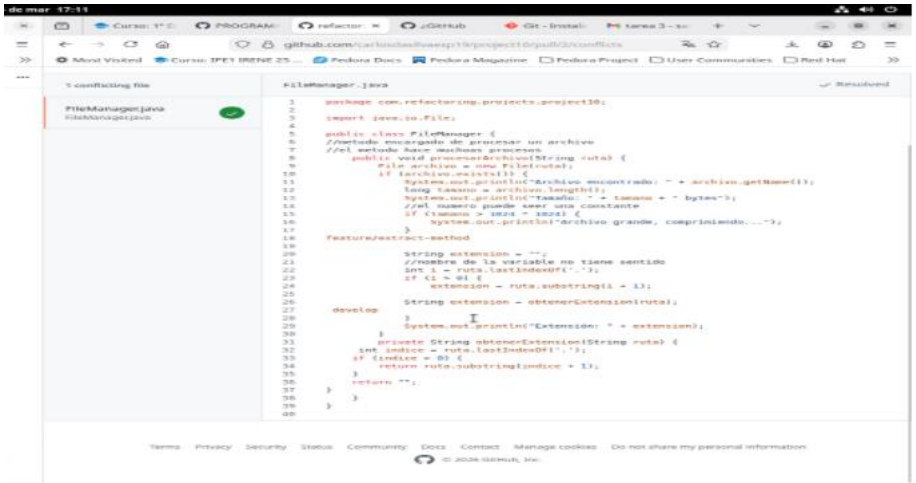
Después de crear el Pull Request para integrar los cambios de la rama feature/extract-method en la rama develop, GitHub detectó un conflicto en el archivo FileManager.java.



Esto ocurre porque existen cambios diferentes en las dos ramas en la misma parte del código. Por ello GitHub pide resolver el conflicto manualmente antes de poder completar la fusión.

Resolución del conflicto

Para resolver el conflicto se revisaron las dos versiones del código dentro del editor de GitHub.

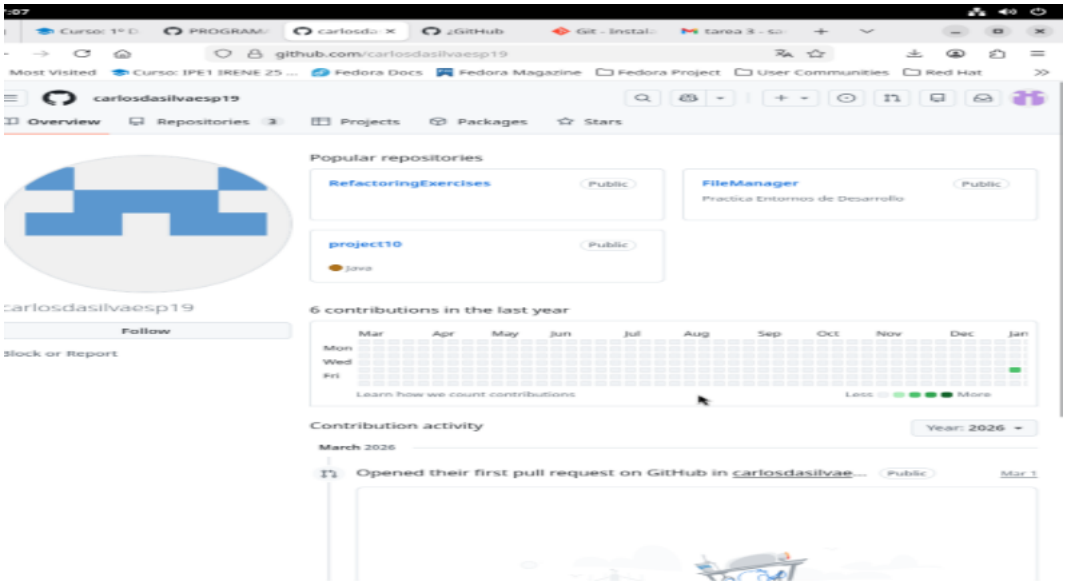


Se decidió mantener la versión refactorizada que utiliza el método: obtenerExtension(ruta). Esta versión mejora la organización del código y hace que el método procesarArchivo sea más limpio y reutilizable. Una vez corregido el conflicto, se marcó el archivo como Resolved.

Confirmación en el repositorio

Una vez resuelto el conflicto y completado el proceso de integración, los cambios quedan registrados en el repositorio.

En el perfil de GitHub se puede ver la actividad donde aparece que se ha abierto y gestionado un Pull Request dentro del proyecto.



El objetivo de esta práctica ha sido aprender a utilizar Git y GitHub para la gestión de versiones de un proyecto de software, trabajando con ramas, commits y Pull Requests. También se ha buscado aplicar técnicas de refactorización para mejorar la calidad y organización del código.

Gracias a esta, se ha comprendido cómo funcionan las herramientas de control de versiones en un entorno de desarrollo real. Se ha aprendido la importancia de trabajar con ramas, revisar código antes de integrarlo y mantener un historial claro de cambios para facilitar el trabajo en equipo.